STONELINK RISK SIMULATION COCKPIT

Mathematical Foundations, Risk Ingestion & System Architecture Documentation

1. System Architecture & General Organization

The Stonelink Risk Simulation Cockpit is a high-performance, full-stack financial simulator designed to evaluate the long-term survival probability of multi-asset investment portfolios under varying market conditions. The application separates concerns cleanly between a decoupled React user interface and an analytical Django REST backend.

Component	Files Involved	Key Responsibilities
Backend API view	`backend/simulation/views.py`	Handles POST requests, extracts payload variables, triggers calculation threads, and enforces reproducibility controls.
Simulation Engine	`backend/simulation/engine.py`	Vectorized NumPy engine performing multivariate Monte Carlo walks, Student-t stress branching, and spectral matrix repair.
Ingestion Pipeline	`backend/simulation/ingestion.py`	Reads, cleans, and structures expected returns, volatilities, and covariance coefficients from Excel files.
Frontend Dashboard	`frontend/src/App.jsx`	Presents the telemetry dashboard, config form, passive seed toggles, and handles asynchronous dashboard updates.
Style System	`frontend/src/index.css`	Vanilla CSS rules implementing cockpit layout grids, neon dashboard lights, and tactility response rules.

2. Ingestion & Data Preparation

The ingestion module dynamically processes the data sheet in *portfolio_data.xlsx*, locating the parameters for 18 Indian asset classes (returns, volatilities, and asset class tags), the correlation matrix, and portfolio allocation weights. The module extracts column headers containing newline characters (e.g. *Portfolio 1\n(Min Risk)*) and maps them to short dropdown tags in the React frontend (e.g. *Min Risk*). Additionally, a **1.4x NAV Unsmoothing Factor** is applied to all private assets to scale up their historical volatilities, adjusting for appraisal lags.

3. Spectral Covariance Matrix Repair Math

A correlation matrix must be positive semi-definite (PSD) to compute its Cholesky decomposition factor. Stale valuation data, appraisal lags, or inconsistent historical periods can cause the parsed matrix to carry negative eigenvalues, rendering standard simulations impossible. The engine detects and repairs this dynamically via spectral decomposition:

> *Let Sigma be the correlation matrix. We calculate its eigenvalues Lambda and eigenvectors Q:*
>
> *Sigma = Q * Lambda * Q^T*
>
> *We force any negative eigenvalues to a small positive floor epsilon (10^-8):*
>
> *Lambda'_i = max(Lambda_i, 10^-8)*
>
> *We then reconstruct the adjusted matrix Sigma':*
>
> *Sigma' = Q * Lambda' * Q^T*
>
> *Finally, to restore unity on the diagonals, we normalize the correlation coefficients:*
>
> *Sigma'_{i,j} = Sigma'_{i,j} / sqrt(Sigma'_{i,i} * Sigma'_{j,j})*

This repaired correlation matrix is then decomposed into its lower-triangular Cholesky factor **L** such that **L * L^T = Sigma'**, which is used to inject correlation into independent random standard normal draws.

4. Asset-Class Specific Simulation Math

The simulation maps M trials over N years. For each trial and year, we draw a vector of independent standard normal random variables Z ~ N(0, I) and project correlation via **X = Z * L^T**.

4.1. STANDARD_CRUISE Mode (Lognormal Walk)

In Cruise Mode, all assets are simulated as standard normal variables scaled by returns and volatilities. For each asset i in year t, the simulated return R_t is calculated as:

```
R_t = expected_return_i + X_i * vol_adjusted_i
```

Where the vol_adjusted_i equals the parsed asset volatility, scaled by 1.4 if the asset class is categorized as 'private'.

4.2. MARKET_STRESS Mode (Student-t Distribution & Crash Premium)

To model joint extreme tail-risks, when Stress Test is enabled, liquid public equities are simulated using a **Multivariate Student-t Distribution (df = 4)**, which introduces fat tails and simultaneous shocks. Furthermore, a **-2.0% pa crash premium** is subtracted from equities. Bonds and private assets remain normal to maintain their yield stability structure while subject to the correlated equity shock:

```
Generate a Chi-Squared random variable: W ~ Chi^2(df = 4)

Compute scale factors: S = sqrt(W / 4.0)

For each asset j:

If class == 'equity': X_j = (X_j / S) - 0.02

If class != 'equity': X_j = X_j
```

This introduces severe simultaneous downsizes for equities while keeping fixed-income linear yields predictable.

5. Portfolio Accumulation, Spending & Capital Preservation

The portfolio value V_t at year t accumulates inflows and withdraws inflation-adjusted spending. Let V_0 be the initial value. For year t = 1 to N:

```
1. Add savings contribution (if t <= retirement_year):

V'_{t} = V_{t-1} + C_0 * (1 + contribution_growth)^(t-1)

2. Apply growth factors based on allocations (w_i):

V''_{t} = V'_{t} * (1 + sum_i(w_i * R_{i,t}))

3. Deduct inflation-adjusted withdrawals (if t > retirement_year):

V_{t} = max(V''_{t} - W_0 * (1 + inflation_rate)^(t-1), 0)
```

Capital Preservation Success Hurdle: A trial is a success if V_t is greater than or equal to a target hurdle (defaulting to the initial portfolio value V_0). Under **Nominal View**, the comparison is made directly on V_t. Under **Real View**, the values are deflated by the inflation rate before comparison: *V_real_t = V_t / (1 + inflation_rate)^t*. The success rate is the percentage of trials exceeding the hurdle at year t.

6. Concurrency, Reentrancy & Seed Control

To provide a reproducible baseline, the backend accepts a boolean *use_fixed_seed*. To avoid concurrency collisions where simultaneous requests interleave and overwrite process-wide global seeds (like *np.random.seed(42)*), the engine instantiates an isolated local Random Generator instance for each calculation request:

```
seed_value = 42 if use_fixed_seed else None

rng = np.random.default_rng(seed_value)

Z = rng.normal(...) and W = rng.chisquare(...)
```

This isolated *rng* design guarantees reentrancy and thread-safety under active concurrent server environments like PythonAnywhere, keeping simulations perfectly reproducible on click repetitions without race conditions.